1.Menu Pausa

```
3 referencias
private void EstablecerPausa(bool nuevaPausa)
{
    /* cambia el estado de pausa */
    pausado = nuevaPausa;
    //menu de pausa invisible
    float destino = 0f;//destino de opacidad del panel de pausa
    //menu de pausa visible
    if (pausado) { Time.timeScale = 0f; destino = 1f; }
    else { Time.timeScale = 1f; }
    panelPausa.interactable = pausado;
    panelPausa.blocksRaycasts = pausado;
    if (transicion != null) { StopCoroutine(transicion); }
    transicion = StartCoroutine(HacerFade(destino));
    if (!pausado && EventSystem.current != null)
    { EventSystem.current.SetSelectedGameObject(null); }
}
```

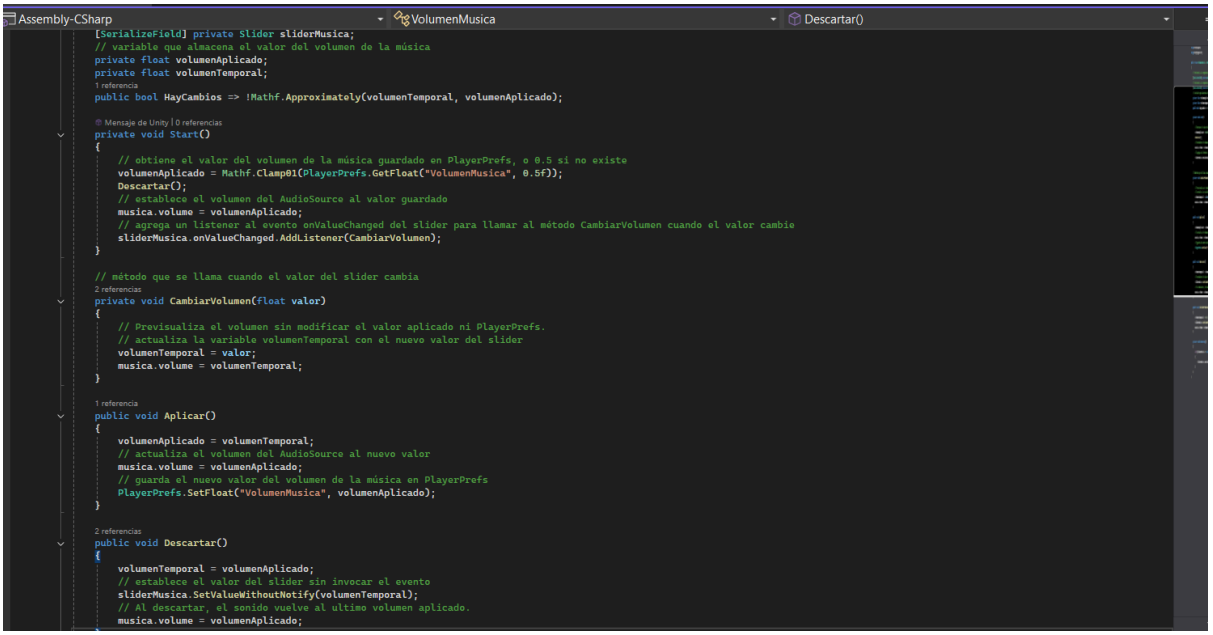
Pausa del juego: el menú modifica `Time.timeScale` a 0 al pausar y a 1 al continuar. También inicia una transición de transparencia mediante una corrutina.

1.1 fade de pantalla

```
//Cambia la opacidad del panel de pausa para crear un efecto de transición suave
1 referencia
private IEnumerator HacerFade(float destino)//metodo para hacer un fade in o fade out del panel de pausa
{
    float inicio = panelPausa.alpha;
    float tiempo = 0f;
    //mientras el tiempo transcurrido sea menor que la duración del fade
    while (tiempo < duracionFade)
    {
        tiempo += Time.unscaledDeltaTime;
        panelPausa.alpha = Mathf.Lerp(inicio, destino, tiempo / duracionFade);
        //espera un frame antes de continuar con la siguiente iteración
        yield return null;
    }
    panelPausa.alpha = destino;
    transicion = null;
}
```

una corrutina modifica el alpha del Canvas Group para crear el fade de entrada y salida. Utiliza tiempo no escalado para funcionar durante la pausa. (El menú aparece y desaparece progresivamente mediante el **Canvas Group**. `Time.unscaledDeltaTime` permite que la transición funcione aunque el juego esté pausado.)

2. Slider musica



```
Assembly-CSharp | VolumenMusica | Descartar()

[SerializeField] private Slider sliderMusica;
// variable que almacena el valor del volumen de la música
private float volumenAplicado;
private float volumenTemporal;
1 referencia
public bool HayCambios => !Mathf.Approximately(volumenTemporal, volumenAplicado);

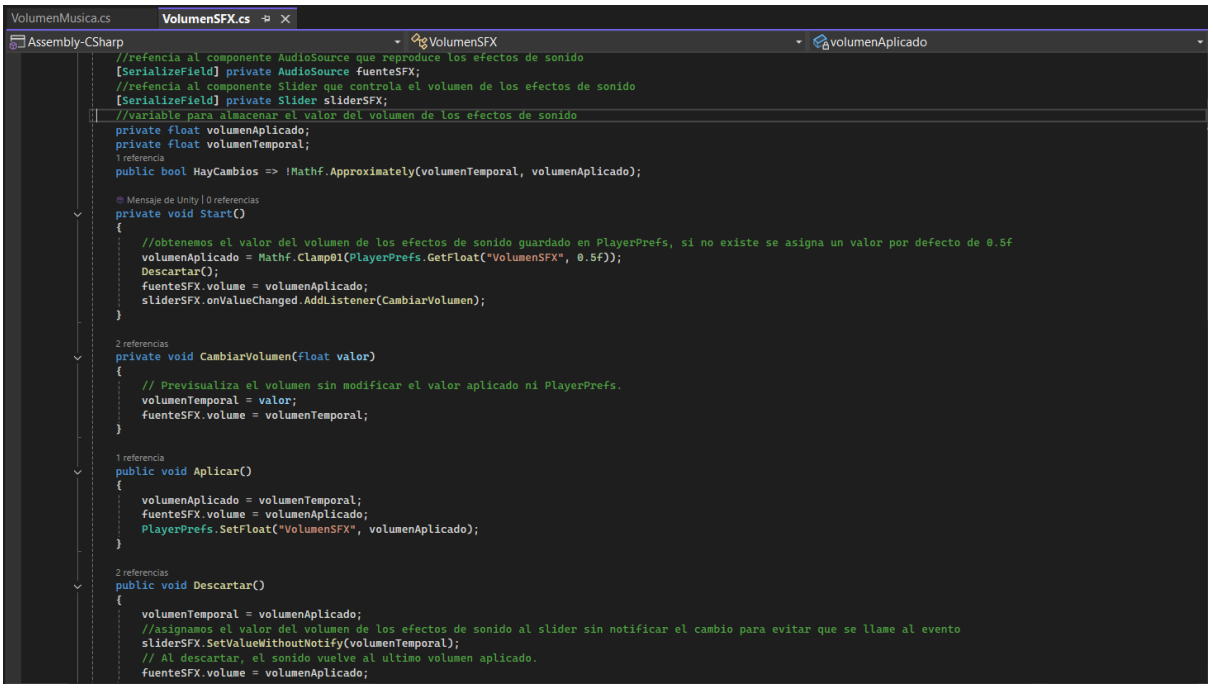
Mensaje de Unity | 0 referencias
private void Start()
{
    // obtiene el valor del volumen de la música guardado en PlayerPrefs, o 0.5 si no existe
    volumenAplicado = Mathf.Clamp01(PlayerPrefs.GetFloat("VolumenMusica", 0.5f));
    Descartar();
    // establece el volumen del AudioSource al valor guardado
    musica.volume = volumenAplicado;
    // agrega un listener al evento onValueChanged del slider para llamar al método CambiarVolumen cuando el valor cambie
    sliderMusica.onValueChanged.AddListener(CambiarVolumen);
}

// método que se llama cuando el valor del slider cambia
2 referencias
private void CambiarVolumen(float valor)
{
    // Previsualiza el volumen sin modificar el valor aplicado ni PlayerPrefs.
    // actualiza la variable volumenTemporal con el nuevo valor del slider
    volumenTemporal = valor;
    musica.volume = volumenTemporal;
}

1 referencia
public void Aplicar()
{
    volumenAplicado = volumenTemporal;
    // actualiza el volumen del AudioSource al nuevo valor
    musica.volume = volumenAplicado;
    // guarda el nuevo valor del volumen de la música en PlayerPrefs
    PlayerPrefs.SetFloat("VolumenMusica", volumenAplicado);
}

2 referencias
public void Descartar()
{
    volumenTemporal = volumenAplicado;
    // establece el valor del slider sin invocar el evento
    sliderMusica.SetValueWithoutNotify(volumenTemporal);
    // Al descartar, el sonido vuelve al ultimo volumen aplicado.
    musica.volume = volumenAplicado;
}
```

slider sfx



```
VolumenMusica.cs | VolumenSFX.cs
Assembly-CSharp | VolumenSFX | volumenAplicado

//refencia al componente AudioSource que reproduce los efectos de sonido
[SerializeField] private AudioSource fuenteSFX;
//refencia al componente Slider que controla el volumen de los efectos de sonido
[SerializeField] private Slider sliderSFX;
//variable para almacenar el valor del volumen de los efectos de sonido
private float volumenAplicado;
private float volumenTemporal;
1 referencia
public bool HayCambios => !Mathf.Approximately(volumenTemporal, volumenAplicado);

Mensaje de Unity | 0 referencias
private void Start()
{
    //obtenemos el valor del volumen de los efectos de sonido guardado en PlayerPrefs, si no existe se asigna un valor por defecto de 0.5f
    volumenAplicado = Mathf.Clamp01(PlayerPrefs.GetFloat("VolumenSFX", 0.5f));
    Descartar();
    fuenteSFX.volume = volumenAplicado;
    sliderSFX.onValueChanged.AddListener(CambiarVolumen);
}

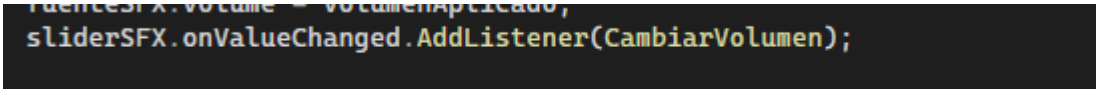
2 referencias
private void CambiarVolumen(float valor)
{
    // Previsualiza el volumen sin modificar el valor aplicado ni PlayerPrefs.
    volumenTemporal = valor;
    fuenteSFX.volume = volumenTemporal;
}

1 referencia
public void Aplicar()
{
    volumenAplicado = volumenTemporal;
    fuenteSFX.volume = volumenAplicado;
    PlayerPrefs.SetFloat("VolumenSFX", volumenAplicado);
}

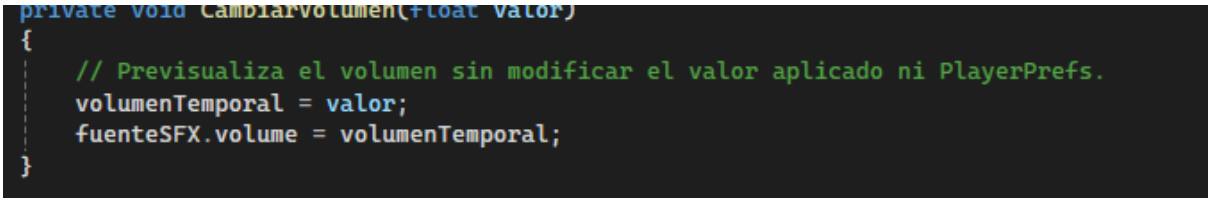
2 referencias
public void Descartar()
{
    volumenTemporal = volumenAplicado;
    //asignamos el valor del volumen de los efectos de sonido al slider sin notificar el cambio para evitar que se llame al evento
    sliderSFX.SetValueWithoutNotify(volumenTemporal);
    // Al descartar, el sonido vuelve al ultimo volumen aplicado.
    fuenteSFX.volume = volumenAplicado;
}
```

los dos sliders modifican variables locales temporales. Al pulsar Apply, los valores se confirman y se guardan en PlayerPrefs. Al iniciar, se recuperan los valores guardados y se establece la posición de cada slider. El guardado se realiza con Apply

3.Sons



```
sliderSFX.onValueChanged.AddListener(CambiarVolumen);
```



```
private void CambiarVolumen(float valor)
{
    // Previsualiza el volumen sin modificar el valor aplicado ni PlayerPrefs.
    volumenTemporal = valor;
    fuenteSFX.volume = volumenTemporal;
}
```

los cambios de volumen se ejecutan mediante el evento `onValueChanged` cuando el usuario mueve el slider, sin actualizar el volumen continuamente en `Update()`.

4.Dropdown

```
Vector2Int guardada = new Vector2Int(
    //obtenemos el ancho guardado en PlayerPrefs, si no existe se asigna el ancho actual de la pantalla
    PlayerPrefs.GetInt("ResolucionAncho", actual.x),
    //obtenemos el alto guardado en PlayerPrefs, si no existe se asigna el alto actual de la pantalla
    PlayerPrefs.GetInt("ResolucionAlto", actual.y));
```

```
2 referencias
public void Descartar()
{
    indiceTemporal = indiceAplicado;
    //establecemos el valor seleccionado sin notificar a los listeners
    dropdownResolucion.SetValueWithoutNotify(indiceTemporal);
    //actualizamos la visualización del dropdown
    dropdownResolucion.RefreshShownValue();
}
```

la resolución se carga desde PlayerPrefs al iniciar. Se recuperan el ancho y el alto guardados y se pone la opción correspondiente en el dropdown. Si la resolución guardada no está disponible, se utiliza el tamaño actual.

4.1 Guardar y probar la resolución

```
1 referencia
public void Aplicar(bool pantallaCompleta)
{
    indiceAplicado = indiceTemporal;
    AplicarAlJuego(pantallaCompleta);
    //obtenemos la resolución seleccionada de la lista
    Vector2Int tamano = resoluciones[indiceAplicado];
    //guardamos el ancho de la resolución seleccionada en PlayerPrefs
    PlayerPrefs.SetInt("ResolucionAncho", tamano.x);
    //guardamos el alto de la resolución seleccionada en PlayerPrefs
    PlayerPrefs.SetInt("ResolucionAlto", tamano.y);
}
```

al pulsar Apply, se confirma la resolución temporal y se guarda el ancho y el alto en PlayerPrefs. Al descartar los cambios, se recupera la opción aplicada anteriormente.

5.Toggle

```
private void Start()
{
    //inicializamos el valor inicial en 0 (modo ventana)
    int inicial = 0;
    //verificamos si la pantalla está en modo completo
    if (Screen.fullScreen) { inicial = 1; }
    //obtenemos el valor guardado en PlayerPrefs, si no existe se asigna el valor inicial
    modoAplicado = PlayerPrefs.GetInt("PantallaCompleta", inicial) == 1;
    Descartar();
    togglePantallaCompleta.onValueChanged.AddListener(CambiarModo);
    // ResolucionMenu restaura resolucion y modo juntos al iniciar.
}
```

```
1 referencia
public void Aplicar()
{
    modoAplicado = modoTemporal;
    //inicializamos el valor guardado en 0 (modo ventana)
    int valor = 0;
    if (modoAplicado) { valor = 1; }
    //guardamos el valor en PlayerPrefs para que se mantenga entre sesiones
    PlayerPrefs.SetInt("PantallaCompleta", valor);
}

2 referencias
public void Descartar()
{
    modoTemporal = modoAplicado;
    togglePantallaCompleta.SetIsOnWithoutNotify(modoTemporal);
}
```

el toggle permite cambiar entre ventana y pantalla completa. Se puede probar el cambio antes de Apply. Al aplicar se guarda el estado como 0 o 1 en PlayerPrefs y al iniciar se carga otra vez, tanto en el toggle como en la aplicación.

6.Responsive UI

Formato 16:9



Formato 16:10



he comprobado el menú en los formatos 16:9 y 16:10. Los controles se mantienen dentro de la pantalla y la interfaz adapta su tamaño al cambiar el formato.

7.EXTRA

```
1 referencia
public void CambiarPausa()
{
    if (!pausado)
    {
        Descartar();
        EstablecerPausa(true);
    }
    else if (panelConfirmacion.activeSelf) { CancelarSalida(); }
    else if (HayCambios) { MostrarConfirmacion(); }
    else { EstablecerPausa(false); }
}
```

cuando intento cerrar el menú con Escape, se comprueba si hay cambios sin aplicar. Si hay cambios, aparece el panel de confirmación y se bloquean los controles de detrás para evitar pulsarlos.

7.1 Botones de confirmación

```
4 referencias
public void CancelarSalida()
{
    panelConfirmacion.SetActive(false);
    for (int i = 0; i < controles.Length; i++)
    { controles[i].interactable = estadosControles[i]; }
    if (EventSystem.current != null)
    { EventSystem.current.SetSelectedGameObject(botonApply.gameObject); }
}
```

```
2 referencias
public void DescartarYSalir()
{
    CancelarSalida();
    Descartar();
    EstablecerPausa(false);
}
```

si pulso No, se oculta el aviso y sigo en el menú con los cambios temporales. Si pulso Sí, se descartan los cambios, se recuperan los últimos valores aplicados y se cierra el menú. La resolución y los volúmenes también vuelven a los valores anteriores.

7.2 Apply y Default Values

```
2 referencias
public void Aplicar()
{
    volumenMusica.Aplicar();
    volumenSFX.Aplicar();
    pantalla.Aplicar();
    resolucion.Aplicar(pantalla.PantallaCompleta);
    PlayerPrefs.Save();
}
```

```
2 referencias
public void ValoresPredeterminados()
{
    volumenMusica.ValoresPredeterminados();
    volumenSFX.ValoresPredeterminados();
    resolucion.ValoresPredeterminados();
    pantalla.ValoresPredeterminados();
    // Prueba ambos valores juntos despues de actualizar los dos controles.
    resolucion.Previsualizar(pantalla.PantallaCompleta);
}
```

Apply llama a los scripts de cada ajuste para confirmar sus valores y PlayerPrefs.Save guarda los cambios. Default Values coloca los valores predeterminados para probarlos, pero hay que pulsar Apply para guardarlos. Los scripts de música, SFX, resolución y pantalla completa están separados.